

AI ADOPTION • FINANCE WORKFLOW • HUMAN-GATED IMPLEMENTATION

Variq - AI Variance Reporting Toolkit

A human-gated finance workbench that turns actual-versus-budget CSV data into deterministic variance analysis and evidence-constrained AI commentary.

CURRENT STATUS

Working local MVP; build and repository tests pass, while persistence and production-grade controls remain planned

EVIDENCE REVIEWED

Evidence basis: repository source, configuration, product specification, visual asset, two commits dated 14 August 2026, and validation rerun on 26 August 2026. No live OpenAI request, hosted deployment or external user workflow was exercised.

Problem

Monthly variance reporting often combines spreadsheet preparation, calculation, materiality decisions, business context, commentary drafting and approval in a loosely controlled process. This creates repeated work and makes it difficult to distinguish calculated facts from explanations that still need evidence or human judgement.

Objective

Build a focused web MVP in which code owns the financial calculations, an AI service may draft only from supplied facts and owner context, and a reviewer retains control over edits, approval and publication. Keep the product specification broader than the prototype without presenting planned database, security or reporting capabilities as already delivered.

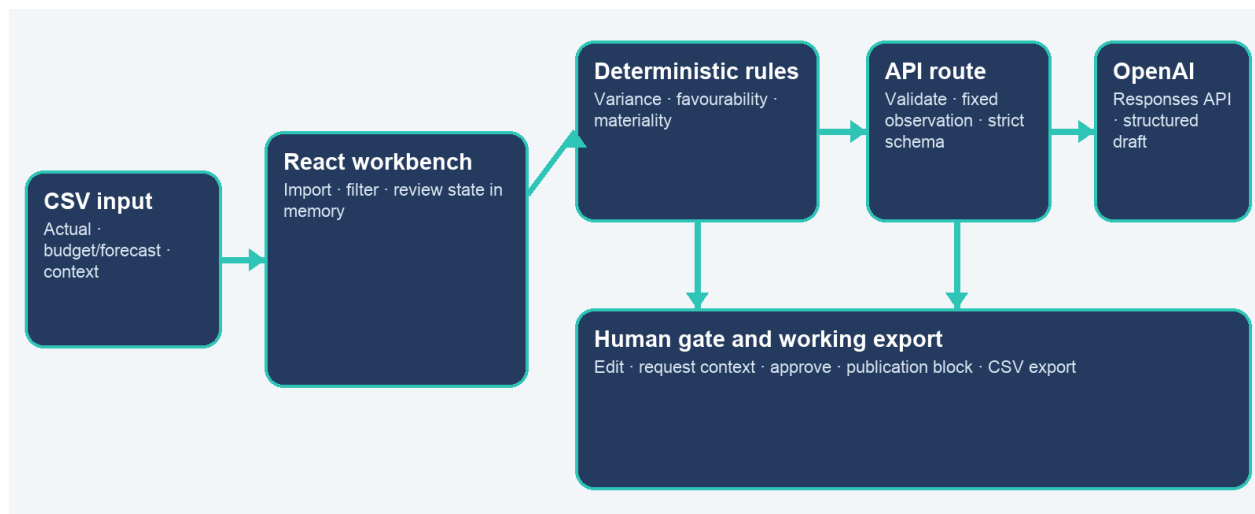
WHY THIS MATTERS

Variq demonstrates practical AI adoption in a high-trust workflow. The model is used for a bounded language task, while calculation, evidence, review and release decisions remain explicit application responsibilities. It also shows the difference between a persuasive product prototype and the controls needed for operational deployment.

SYSTEM VIEW

Architecture and workflow

The current implementation is primarily client-side. React imports and validates CSV rows, calculates variances and materiality, and manages review state in memory. For drafting, the client sends calculated facts and owner context to a server route, which calls the OpenAI Responses API with a strict schema. Vinext packages the Next-style app for a Cloudflare Worker/Sites runtime.



Architecture reconstructed from the reviewed implementation artefacts.

Operational workflow

1. Import a CSV containing Account, Actual and Budget/Forecast headings, with optional department, owner, context and account type fields.
2. Validate required columns and numeric values, retain the source filename and row reference, then replace the sample rows in browser state.
3. Calculate absolute and percentage variance, favourability and materiality with user-adjustable absolute or percentage thresholds.
4. Filter the workbench, select a material item and add attributable business context beside the calculated evidence.
5. Request a structured AI draft; the server constructs the numerical observation and permits causal language only when supplied context supports it.
6. Edit, request context or approve commentary. Publication remains blocked while required items are unapproved.
7. Export the current working rows and commentary to CSV; refresh or a new session does not preserve the workflow state.

FOUNDATIONS

Technologies and controls

The stack below reflects the reviewed implementation; legacy or optional paths are identified explicitly.

AREA	EVIDENCED TECHNOLOGY / PATTERN
Web application	React 19, TypeScript 5.9 and a Next-style app router compiled through Vnext and Vite 8
Edge runtime	Cloudflare Worker entry point, Cloudflare Vite plugin and private Sites project configuration
AI drafting	OpenAI Responses API, gpt-5.4-mini, strict JSON Schema output and store=false
Data workflow	Browser CSV parsing, source-row attribution, deterministic variance/materiality functions and CSV export
Interface	Custom responsive CSS, accessible labels, keyboard-selectable rows, filters, review panel and status feedback
Quality	Node test runner, server-rendered HTML checks, ESLint, TypeScript strict mode and production build
Scaffolding	Drizzle ORM/D1 helper and ChatGPT header-auth utilities are present but are not wired into the current workflow

DELIVERY

What was actually built

Evidence labels separate implemented work from claims, plans and dependencies.

IMPLEMENTED Interactive variance workbench

Overview, analysis, review and audit views are present, with search, status filters, summary cards, configurable thresholds and keyboard-selectable rows.

IMPLEMENTED CSV import and working-report export

The browser validates Account, Actual and Budget/Forecast columns, accepts several aliases, checks numeric values, records source rows and exports the current report to CSV.

IMPLEMENTED Deterministic variance rules

Application functions calculate raw and percentage variance, income/expense favourability and an OR-based absolute/percentage materiality rule.

IMPLEMENTED Evidence-constrained AI endpoint

The server builds the numerical observation itself, sends minimised facts and owner context, requests strict structured output and rejects incomplete requests before provider use.

IMPLEMENTED WITH CAVEAT Human review and publication gate

Users can edit, request context and approve items; the publish action is blocked until required items are approved, but all state and enforcement are currently in browser memory.

IMPLEMENTED AS INTERFACE PROTOTYPE Audit and revision presentation

The UI displays policy versions, events and revision language, but events are predefined and neither an append-only audit store nor audit export is implemented.

SCAFFOLDED, NOT OPERATIONAL Database and authentication foundations

A D1/Drizzle helper and ChatGPT authentication utilities exist; the schema is empty, the D1 binding is null and the page does not require the auth helper.

PLANNED Full product requirements

XLSX ingestion, reconciliation, persistent roles, immutable report versions, background jobs, section approvals, richer exports and operational controls remain specification content rather than current MVP behaviour.

Implementation notes

- The model cannot change actuals, budget values, variance calculations or approval state through the commentary endpoint.
- Causal text is cleaned and discarded if it contains numbers; unsupported context produces an observation plus a reviewer question or client-side fallback.
- Confidence values are fixed bands in the current code, not the weighted, versioned application score described in the product requirements.
- The interface uses sample August 2026 data and the generated observation also hard-codes August, so period handling is not yet generalised.
- The product requirements document is substantially broader than the two-commit prototype and is treated as design intent throughout this review.

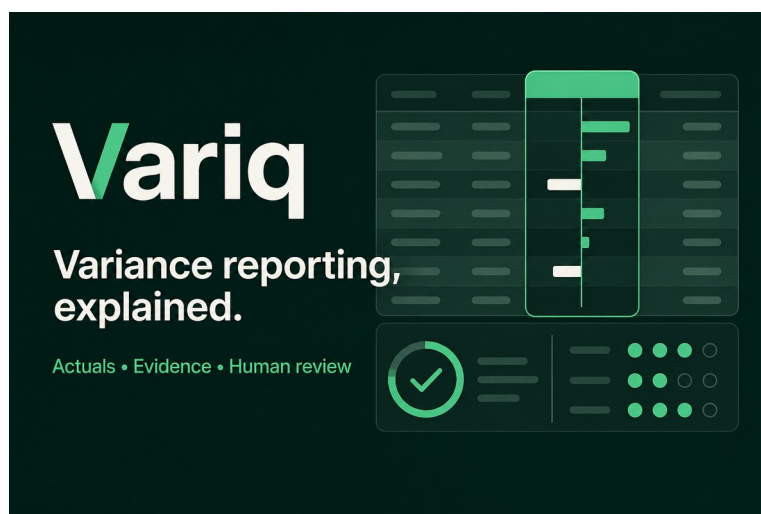
ASSURANCE

Testing and validation evidence

Checks were rerun where the local repository and dependencies made that practical.

CHECK	RESULT	EVIDENCE
Production build	PASS	npm test completed the five-stage Vinext/Vite build successfully on 26 August 2026.
Automated repository tests	PASS	Two Node tests passed: server rendering of the workbench and rejection of an incomplete commentary request before provider use.
Mandatory-gate evidence	PASS (rendered UI)	The server-rendered HTML test confirms the Human approval required message and core variance content are present.
Lint	FAIL	ESLint reports one react-hooks/set-state-in-effect error where selected-row values are copied into two draft state variables.
Live AI and failure modes	NOT TESTED	No request was sent to the external provider, and no automated tests cover schema parsing, supported-cause handling, provider errors or the browser fallback.
Persistence and deployment	NOT VERIFIED	No database-backed workflow is configured and the private Sites deployment was not opened or independently exercised during this review.

Example output



Repository social preview asset. It supports the project's visual identity; it is not evidence of a live deployment or user adoption.

REFLECTION

Limits, lessons and next steps

Limitations

- Data, edits, approvals, audit presentation and publication state are session-only; the audit and publish controls are not persisted or tamper-evident.
- Financial calculations use JavaScript numbers rather than decimal arithmetic and have limited automated coverage.
- The importer supports CSV only; XLSX, mapping previews, reconciliation and immutable source storage remain planned.
- Authentication is not enforced, D1 is unconfigured, and period, currency, confidence and zero-comparison logic are partly hard-coded.
- The live AI call, hosted environment, accessibility conformance and representative finance-data volumes were not independently validated.

Lessons learned

- Keep authoritative calculations outside the language model and pass them into a narrowly scoped drafting service.
- Structured output and attributable context reduce unsupported explanations, but application-side validation still needs comprehensive tests.
- A visible approval button is useful interaction design; durable governance requires the same gate in authenticated server and database transitions.
- A detailed PRD is a useful control target when built, scaffolded and planned capabilities - and both passing and failing checks - remain clearly separated.

Next steps

1. Fix the React effect lint error and expand unit coverage around calculations, CSV parsing, zero budgets, favourability and threshold combinations.
2. Move packs, source versions, context, revisions, approvals and audit events into an authenticated server-side data model.
3. Replace floating-point money logic with governed decimal calculations and make period, currency and comparison settings explicit.
4. Add mocked and controlled integration tests for structured AI output, unsupported causes, provider failures, fallbacks and correlation IDs.
5. Implement server-enforced approval/publication transitions, then verify deployment, accessibility and representative data volumes before broader claims.

Evidence base reviewed

- README, package/configuration files, application, API, worker, database and authentication source
- Repository test plus build and lint validation rerun on 26 August 2026
- Product requirements and build prompt, treated as planned intent where not implemented
- Git commits 12e5dbb and 6b52955, plus the repository social preview asset

CLAIM DISCIPLINE *Implemented claims are restricted to reviewed artefacts; planned or externally dependent items are labelled.*